

AI Operations (AIOps)

Module 1: Experiment Management & Reproducibility

Course Content — 3 Lectures (8–9 Contact Hours) | Third-Year B.Sc./B.Tech. Data Science

Module Overview

This module establishes the conceptual and practical foundation for the entire AIOps course. Students move from the mindset of "train a model in a notebook" to "engineer a model as a governed, auditable, reproducible artifact." The module is delivered across three lecture sessions, each combining theory with hands-on lab work.

Lecture	Title	Duration	Core Focus
L1	MLOps Foundations & Technical Debt	~3.0 hrs	Why AIOps exists; hidden technical debt; MLOps vs DevOps; Git + Conda/Mamba environment setup
L2	Experiment Tracking with MLflow	~3.0 hrs	Logging parameters, metrics, artifacts; MLflow Tracking Server architecture; comparing runs
L3	Reproducibility: Data & Model Versioning	~2.5–3.0 hrs	DVC for data versioning; MLflow Model Registry; regulatory/audit case studies

Module Learning Outcomes

- Explain why production ML systems accrue technical debt faster than traditional software (CO1, CO5).
- Differentiate MLOps from DevOps in terms of data gravity, model decay, and testing surface area (CO1).
- Instrument a training script to log parameters, metrics, and artifacts using MLflow Tracking (CO5).
- Compare and audit experiment runs using the MLflow UI and Model Registry (CO5, CO6).
- Version datasets with DVC and bind a specific data version to a specific Git commit (CO5).
- Design a minimal reproducibility protocol (code + data + environment + model) suitable for a regulated setting such as healthcare (CO6, CO9).

Lecture 1 — MLOps Foundations & Technical Debt (≈3 hours)

1.1 The Production Gap

Most undergraduate deep learning courses end at `model.fit()` and a leaderboard accuracy score. In industry, this is roughly 5–10% of the total engineering effort. The remaining effort — data pipelines, serving infrastructure, monitoring, retraining triggers, rollback plans — is what AIOps (used interchangeably with MLOps in this course) addresses. We call the gap between "a model that works once, on my machine" and "a model that works continuously, for everyone, safely" the Production Gap.

Discussion prompt for class

Ask students: "You trained a 96%-accuracy image classifier for your course project. What would have to be true for a hospital to actually use it on real patients tomorrow?" Expected answers to draw out: monitoring, versioning, rollback, explainability, latency guarantees, retraining pipeline, audit trail.

1.2 Hidden Technical Debt in Machine Learning Systems

This section is grounded in the seminal industry framing (Sculley et al., "Hidden Technical Debt in Machine Learning Systems") that ML systems incur a form of technical debt that is harder to detect and pay down than debt in ordinary software, because the debt often lives in data relationships rather than in code.

Key sources of ML technical debt

- **Boundary Erosion** — ML systems mix statistical logic with rule-based logic; clean abstraction boundaries used in classical software engineering break down.
- **Entanglement (the CACE principle)** — "Changing Anything Changes Everything." Altering one feature, hyperparameter, or upstream data source can silently shift the behaviour of the whole model.
- **Hidden Feedback Loops** — A model's own predictions influence the future data it is trained on (e.g., a recommender that shapes user behaviour, which then shapes its own training data).
- **Undeclared Consumers** — Other teams silently consume a model's output (e.g., via a shared table), creating invisible dependencies that block safe changes.
- **Data Dependencies** — Unstable data dependencies (upstream data whose meaning or distribution changes without warning) are more dangerous and harder to detect than unstable code dependencies.
- **Configuration Debt** — Hyperparameters, feature selections, and versions scattered across scripts, notebooks, and config files with no single source of truth.
- **Glue Code and Pipeline Jungles** — Systems evolve into a tangle of scripts stitching together data extraction, transformation, and model calls, with no clean orchestration layer.

Why this matters for AIOps

Every remaining module in this course is, in effect, a response to one of these debt categories: experiment tracking counters configuration debt; orchestration (Airflow/Kubeflow) counters pipeline jungles; feature stores counter undeclared consumers and unstable data dependencies; monitoring counters hidden feedback loops and data drift.

1.3 MLOps vs. DevOps

Dimension	DevOps	MLOps / AIOps
What is versioned	Code only	Code + Data + Model + Environment
Testing	Unit / integration tests, deterministic	Statistical validation; non-deterministic outputs; data validation
What degrades in production	Rarely, unless code changes	Model decay even with unchanged code, due to real-world drift
Primary failure mode	Code bugs, downtime	Silent quality degradation (no crash, just wrong answers)
Rollback unit	A code deployment	A code + data + model + config bundle
Core new discipline	CI / CD	CI / CD / CT (Continuous Training)

Data Gravity

Data gravity describes how large datasets attract compute, tooling, and pipelines toward wherever the data physically lives, because moving large data repeatedly is slow and costly. This is why distributed processing (Module 2: Spark, Ray) tends to run close to storage rather than pulling data to a single workstation.

Model Decay

A model's real-world performance degrades over time even if its code and weights never change, because the world it was trained on keeps changing. Two named forms of decay recur throughout this course:

- Data Drift — the statistical distribution of input features shifts (e.g., new customer demographics).
- Concept Drift — the relationship between inputs and the target label itself changes (e.g., "fraud" patterns evolve as fraudsters adapt).

1.4 Case Study Discussion — "It Worked on My Machine"

A student trains a model that reaches 91% validation accuracy locally. When a teammate reruns the identical script on their laptop, they get 84%, and on the department's GPU server, the job crashes outright.

Guided root-cause analysis (walk through in class)

1. Unpinned dependency versions — a newer version of a library changed a default hyperparameter or numerical algorithm.
2. Non-deterministic operations — GPU non-determinism, unseeded random shuffles, unseeded weight initialization.
3. Silent data differences — a "latest" data file that was actually re-exported between runs, changing row order or content.
4. Hardware/driver mismatch — different CUDA/cuDNN versions altering floating-point behaviour.
5. Undocumented manual steps — a one-off notebook cell that patched the data, never captured in the pipeline.

The engineering fix (previewed here, built hands-on in this lecture and Lecture 3)

- Pin the code with Git (exact commit hash).

- Pin the environment with Conda/Mamba (a lockfile-like environment.yml) or a Dockerfile (Module 3).
- Pin the data with DVC, tied to that same Git commit (Lecture 3).
- Pin the run's parameters/metrics/artifacts with MLflow (Lecture 2).

This four-part pinning — code, environment, data, experiment — is the reproducibility protocol we build across all three lectures of this module.

1.5 Hands-on Lab: Git + Conda/Mamba Reproducible Environment

Objective: every student leaves with a Git repository and a Conda/Mamba environment file that can recreate their Python environment exactly on another machine.

Step 1 — Initialize a Git repository

```
git init aiops-module1
cd aiops-module1
git config user.name "Your Name"
git config user.email "you@example.com"
```

Step 2 — Create a Conda/Mamba environment

```
mamba create -n aiops-m1 python=3.10 -y
mamba activate aiops-m1
mamba install pytorch cpuonly mlflow pandas scikit-learn -c pytorch -c conda-forge -y
```

Step 3 — Export a reproducible environment file

```
mamba env export --no-builds > environment.yml
# Cross-check: a pip-only alternative
pip freeze > requirements.txt
```

Step 4 — Commit code and environment together

```
git add environment.yml requirements.txt train.py
git commit -m "Initial reproducible training environment"
```

Step 5 — Verify reproducibility (pair exercise)

Each student pushes their repo to a shared classroom Git remote (or exchanges a zip). Their partner clones it into a fresh Conda environment using only environment.yml, reruns train.py, and confirms they obtain the same metric (within a documented random-seed tolerance).

```
mamba env create -f environment.yml -n aiops-m1-verify
mamba activate aiops-m1-verify
python train.py --seed 42
```

Common failure students hit

mamba env export often captures OS-specific build strings that break cross-platform installs. Using `--no-builds` (as above) or exporting a minimal `--from-history` file is the practical fix — worth demonstrating live.

Lecture 2 — Experiment Tracking with MLflow (≈3 hours)

2.1 The Reproducibility Crisis in Applied ML

A typical practitioner runs dozens to hundreds of experiments while tuning a model: different learning rates, architectures, feature sets, and random seeds. Without systematic tracking, teams lose the ability to answer simple questions: Which run produced the model in production? What hyperparameters produced our best F1 score? Which dataset version was used? Experiment tracking directly counters the Configuration Debt introduced in Lecture 1.

2.2 MLflow Architecture

MLflow is an open-source platform with four main components; this module focuses on MLflow Tracking and the MLflow Model Registry (used in Lecture 3).

Component	Role
Tracking Server	REST/UI service that receives and serves logged run data
Backend Store	Stores metadata: parameters, metrics, tags, run status (e.g., a database or local filesystem)
Artifact Store	Stores large binary outputs: model files, plots, checkpoints (e.g., local disk, S3-compatible storage)
Model Registry	A versioned, stage-aware catalogue of registered models (covered in Lecture 3)

2.3 Core MLflow Tracking Concepts

- **Experiment** — a named collection of related runs (e.g., "chest-xray-classifier").
- **Run** — a single execution of training/evaluation code; the atomic unit of tracking.
- **Parameters** — static input configuration for a run (learning rate, batch size, model architecture).
- **Metrics** — values that can be logged over time/steps (loss per epoch, validation accuracy).
- **Artifacts** — files produced by a run (saved model weights, confusion-matrix plots, sample predictions).
- **Tags** — free-form key/value metadata for filtering and organization (e.g., team, git commit hash).

2.4 Hands-on Lab: Logging a PyTorch Training Run

Step 1 — Start a local MLflow Tracking Server

```
mlflow server --backend-store-uri sqlite:///mlflow.db --default-artifact-root
./mlruns --host 0.0.0.0 --port 5000
```

Step 2 — Point the client at the server and set the experiment

```
import mlflow
mlflow.set_tracking_uri("http://localhost:5000")
mlflow.set_experiment("chest-xray-classifier")
```

Step 3 — Instrument the training loop

```
with mlflow.start_run(run_name="resnet18-lr0.001"):
    mlflow.log_param("lr", 0.001)
    mlflow.log_param("batch_size", 32)
    mlflow.log_param("architecture", "resnet18")
    mlflow.set_tag("git_commit", get_git_commit_hash())
    for epoch in range(num_epochs):
        train_loss = train_one_epoch(model, optimizer, train_loader)
        val_acc = evaluate(model, val_loader)
        mlflow.log_metric("train_loss", train_loss, step=epoch)
        mlflow.log_metric("val_accuracy", val_acc, step=epoch)
    mlflow.pytorch.log_model(model, "model")
    mlflow.log_artifact("confusion_matrix.png")
```

Step 4 — Run multiple experiments and compare

Students repeat the run with at least two different learning rates and two different architectures (e.g., resnet18 vs. a simple CNN), producing 4+ logged runs.

Step 5 — Explore the MLflow UI

- Open <http://localhost:5000> and locate the experiment.
- Use the run comparison view (select multiple runs → Compare) to plot val_accuracy vs. lr.
- Sort the runs table by val_accuracy to identify the best-performing configuration.
- Open a run's Artifacts tab to inspect the saved model and confusion matrix.

Instructor note

Emphasize that `mlflow.pytorch.log_model` saves the model in MLflow's standard "flavor" format, which packages the model with its dependency environment — this is what makes Lecture 3's Model Registry promotion workflow possible.

2.5 Interpreting Comparative Results (in-class exercise)

Provide students a pre-populated MLflow run table (screenshot or shared server) with 6 runs varying lr and batch_size. In pairs, students must identify: (a) the best run by val_accuracy, (b) whether there is evidence of overfitting from the train_loss vs. val_accuracy trend, and (c) which hyperparameter appears to matter more. This builds the analytical habit needed before Module 4 (CI/CD model sanity checks).

Lecture 3 — Reproducibility: Data & Model Versioning (≈2.5–3 hours)

3.1 The Version Control Triad

Reproducibility in ML requires three things to be versioned together, not just code:

What	Tool	Why Git alone fails
Code	Git	—
Data	DVC (Data Version Control)	Datasets are large binary files; Git repositories are not designed to store multi-GB files efficiently
Model	MLflow Model Registry	Models need a lifecycle (staging → production → archived), not just a file version

3.2 Data Versioning with DVC

DVC works alongside Git: instead of committing a large dataset file directly, Git tracks a small human-readable .dvc pointer file (containing a content hash), while the actual data lives in a separate DVC remote (e.g., local disk, S3-compatible bucket, or Google Drive).

Core DVC workflow

1. `dvc init` inside an existing Git repository.
2. `dvc add data/train.csv` creates `data/train.csv.dvc` and adds the real file to `.gitignore` automatically.
3. `git add data/train.csv.dvc .gitignore` && `git commit -m "Track v1 of training data"`.
4. `dvc remote add -d storage s3://my-bucket/dvcstore` (or a local path for a classroom setting).
5. `dvc push` uploads the actual data to the remote; `dvc pull` retrieves it on another machine.

Hands-on Lab: DVC remotes and linking data to a Git commit

```
dvc init
dvc remote add -d localstore /mnt/shared/dvcstore
dvc add data/train_v1.csv
git add data/train_v1.csv.dvc .gitignore
git commit -m "v1 dataset"
dvc push
# --- team edits the dataset, creating a new version ---
dvc add data/train_v1.csv
# re-run after editing the underlying file
git add data/train_v1.csv.dvc
git commit -m "v2 dataset: added 5k new labeled samples"
dvc push
# --- reproduce an OLD experiment exactly ---
```



```
git checkout <commit-hash-of-v1>
dvc checkout
# pulls the exact data version matching that commit
```

The key idea to emphasize: git checkout moves the pointer file back in time; dvc checkout then fetches the exact bytes of data that pointer refers to. Code and data version together, always.

3.3 Model Versioning with the MLflow Model Registry

Once a training run produces a satisfactory model (Lecture 2), it can be registered — promoted from a disposable run artifact to a first-class, versioned entity with a governed lifecycle.

Registry stages

Stage	Meaning
None	Registered but not yet reviewed
Staging	Candidate under validation (e.g., shadow testing, QA)
Production	Currently serving live traffic
Archived	Retired; kept for audit/rollback history

```
import mlflow
result = mlflow.register_model(model_uri="runs:/<run_id>/model", name="chest-xray-classifier")
from mlflow.tracking import MlflowClient
client = MlflowClient()
client.transition_model_version_stage(name="chest-xray-classifier", version=result.version,
stage="Staging")
```

Every stage transition, along with who performed it and when, is retained in the registry's history — this history is precisely what an auditor requests in the case study below.

3.4 Case Study 1: Auditing a Healthcare Model's Training History for Regulatory Compliance

Scenario: A hospital deploys an ML model that flags patients at risk of sepsis. A regulator (or internal clinical safety board) asks: "Show us exactly what data, code, and parameters produced the model version currently running in the ICU, and show every version that has ever been in production."

Guided in-class walkthrough — what must be answerable

- Which exact Git commit produced this model? → git log / MLflow run tag git_commit.
- Which exact dataset version was used, and can we retrieve those exact rows today? → the .dvc pointer committed alongside that code, plus dvc checkout.
- What hyperparameters and metrics did this run report? → MLflow Tracking run page.
- Has this exact model version ever been in Production before, and when did it transition stages? → MLflow Model Registry stage-transition history.
- Who approved the transition to Production, and is that logged? → Registry activity log / comments (governed further in Module 6).

Instructor emphasis: this case study is a preview of Module 6 (Governance & Compliance) — Module 1 supplies the raw traceability primitives (Git commit + DVC data hash + MLflow run + Registry stage history) that governance processes are built on top of.

3.5 Case Study 2: Resolving "It Worked on My Machine" with Environment Versioning

Revisit the Lecture 1 scenario now that students have the full toolset. In groups, have students map each root cause identified in Lecture 1 to the specific artifact from Lectures 1–3 that would have prevented it.

Root Cause (from L1)	Prevented By
Unpinned library versions	environment.yml / requirements.txt committed to Git
Non-deterministic ops / unseeded runs	mlflow.log_param("seed", ...) + fixed seeds in code
Silent data differences	DVC data hash tied to a Git commit
Hardware/driver mismatch	Documented environment + (previewed) containerization in Module 3
Undocumented manual steps	MLflow run tags/notes + code as the single source of truth

3.6 Hands-on Lab: End-to-End Reproducibility Drill

Capstone exercise for the module. Each student/pair performs the full protocol end-to-end:

1. Train a small model, logging the run with MLflow (params, metrics, artifacts, seed, git commit tag).
2. Version the training dataset with DVC and commit the .dvc file alongside the training code in the same Git commit.
3. Register the resulting model in the MLflow Model Registry and transition it to "Staging".
4. Hand their repository URL (or bundle) to a partner, who must reproduce the exact metric using only: git clone, git checkout <commit>, dvc checkout, mamba env create -f environment.yml, and the training script.
5. Partner logs a note in the MLflow run UI confirming reproduction succeeded (or documents the discrepancy).

This drill is directly assessed in the Module 1 assignment (see Assignments document) and forms the rubric baseline the TA uses for grading.

References for This Module

- Sculley, D. et al. "Hidden Technical Debt in Machine Learning Systems." NeurIPS, 2015. (Conceptual basis for Section 1.2.)
- Raj, Emmanuel. Engineering MLOps. Packt Publishing. — Primary text, Chapters on experiment tracking and reproducibility.
- MLflow official documentation: mlflow.org/docs
- DVC official documentation: dvc.org/doc